

SQL RELATIONSHIP

WHAT IS A RELATIONSHIP IN SQL

A **relationship in SQL** describes how two or more tables in a database are connected to each other.

COMMON TYPES OF RELATIONSHIP:

ONE-TO-ONE (1:1):

One record in Table A is related to exactly one record in Table B.

Example:

One person → one passport.

ONE-TO-MANY (1:N):

One record in Table A can relate to many records in Table B.

Example:

One customer → many orders.

MANY-TO-MANY (M:N):

Many records in Table A can relate to many records in Table B.

Example:

Many students → many courses.

This is usually implemented using a **junction/bridge table** such as StudentCourses.

JOIN

The JOIN keyword in SQL is responsible for this kind of work, combining different tables together. So let's start learning how to use this tool!

1.LEFT JOIN:

Returns **all records from the left table**, even if there is no match.

EXAMPLE:

```
SELECT customers.name, orders.amount  
  
FROM customers  
  
LEFT JOIN orders  
  
ON customers.id = orders.customer_id;
```

RESULT:

```
John  500  
  
John  300  
  
Alice NULL
```

2.RIGHT JOIN:

Returns **all records from the right table**, even if there is no match.

Example:

```
SELECT customers.name, orders.amount  
  
FROM customers
```

RIGHT JOIN orders

ON customers.id = orders.customer_id;

RESULT:

name department John

IT

Alice HR

NULL Finance

3.INNER JOIN:

INNER JOIN returns only rows where there is a match in BOTH tables.

EXAMPLE:

SELECT

e.emp_id,

e.name,

d.department

FROM employees e

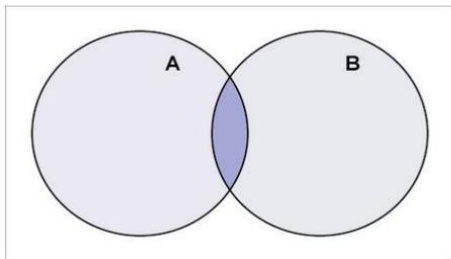
INNER JOIN departments d

ON e.dept_id = d.dept_id;

RESULT:

emp_id_name department

- 1 John IT
- 2 Alice HR



SQL syntax:

```
SELECT * FROM `new_schema`.`users`  
INNER JOIN `new_schema`.`orders` ON `users`.`id` = `orders`.`user_id`;
```

SUBQUERIES

In MySQL, a **subquery** is a query written inside another SQL query. It's useful when the result of one query is needed by another.

EQUAL CONDITION:

First of all, let's take a look at a SQL statement used to address the above situation:

```
SELECT * FROM `new_schema`.`orders`  
WHERE user_id = (  
    SELECT id FROM `new_schema`.`users`  
    WHERE name = 'John'  
);
```

RESULT:

id	user_id	note
1	1	some information

CONTAIN CONDITION:

In addition to the determination of equality, we can also change the SQL into a statement that can solve the condition that contains lots of data.

```
SELECT * FROM `new_schema`.`orders`  
WHERE user_id IN (  
    SELECT id FROM `new_schema`.`users`  
    WHERE name LIKE '%j%'  
);
```

Just like the previous equal example, we can first split the above SQL statement into:

```
SELECT id FROM `new_schema`.`users`  
WHERE name LIKE '%j%'
```

RESULT:

id
1
3